PHC Integrated Digital System for IIT Jodhpur: Solution Trajectory, Implementation Results, and Current Status

Bavadiya Yash Kiritbhai
B24CS1019

Chaitanya Singh
B24CS1088

Nayan Patidar
B24CS1047

Abstract—This report presents the Assignment 2 status of the PHC Integrated Digital System being developed for the Primary Health Centre at IIT Jodhpur. The project targets a campus-scale healthcare environment where patient registration, visit handling, prescriptions, laboratory requests, pharmacy billing, attendance monitoring, and administrative reporting were historically fragmented across paper records and disconnected manual processes. Building on the earlier SRS and UML work, the current implementation has converged on a modular Node.js and Express backend backed by PostgreSQL through Prisma, with role-based access control, JWT session handling, local LDAP-backed authentication for development, QR-assisted check-in, visit-linked traceability, and an expanding automated testing workflow. The report documents the problem motivation, the evolution of the solution from successive SRS refinements, the sprint and Alpha-driven development process, the finalized architectural choices, and the results observed from the working backend. It also records negative findings and operational constraints, including frontend incompleteness, production-readiness gaps, and documentation synchronization drift. The intent is to present not only what the team planned, but what the repository demonstrably achieves as of April 5, 2026.

Index Terms—Healthcare Information Systems, PHC Digitization, QR Check-in, RBAC, Visit Traceability, Agile Scrum, Prisma, PostgreSQL.

I. Problem Statement

A. Objective and Motivation

The IIT Jodhpur Primary Health Centre (PHC) serves a bounded but operationally complex campus population of students, faculty, and staff. In its manual form, the PHC workflow suffers from four persistent problems:

- Fragmented records: patient history, prescriptions, and reports are difficult to connect longitudinally when stored across physical files.
- High reception latency: repeated demographic lookup and visit registration increase waiting time for every walk-in.
- Low operational visibility: doctor availability, attendance, and downstream pharmacy or lab status are not visible in real time.
- Weak traceability: the consultation, prescription, laboratory, and billing artifacts of a single encounter are not reliably tied together.

The objective of the PHC Integrated Digital System is therefore to digitize the entire outpatient lifecycle, from authentication and patient identification to consultation closure and administrative reporting. The system is intended to remain small enough for institutional deployment while still enforcing medical-data discipline comparable to larger healthcare information systems.

The motivation is practical rather than purely academic. The team sought a system that can reduce manual overhead at the reception desk, ensure controlled access to sensitive records, and preserve a single visit-linked source of truth for every medical interaction. The project also had to fit the operating realities of an institute PHC: campus identity infrastructure, limited deployment complexity, a small development team, and the need for iterative validation.

II. Solution Trajectory

A. Evolution from Assignment 1 and SRS Iterations

Assignment 1 captured the project mainly as a design and planning exercise. At that stage, the emphasis was on SRS stabilization, UML modeling, and architectural justification. The current repository shows the transition from that conceptual baseline to a largely implemented backend.

The solution trajectory can be summarized through the main design decisions that survived refinement:

- SRS 1.x to 2.2: the requirements matured from a broad PHC idea to a scoped outpatient system with explicit REQ identifiers, role boundaries, sprint logic, and later testing / validation refinement.
- Document-centric ideas to relational enforcement: earlier ideation explored more flexible storage options, but the final architecture adopted PostgreSQL so that visit-linked integrity could be guaranteed through relations rather than convention.
- Module pruning: non-core ideas such as ambulance-related scope were removed, allowing the system to focus on the PHC's highest-frequency workflows.
- Visit-centric linkage: the SRS concept of a unique VisitID matured in implementation as a visit-centered relational backbone in Prisma. Prescriptions, lab requests, bills, vitals, and documents now attach to a specific visit record.

- Operational differentiation of doctors: the design retained the distinction between SPECIALIST and PHYSICIAN, supporting both scheduled and walk-in driven interactions.

This trajectory matters because the repository does not merely mirror the wording of the SRS. It reflects the points where the team had to refine, simplify, or formalize the original ideas so they could be implemented without losing traceability or maintainability.

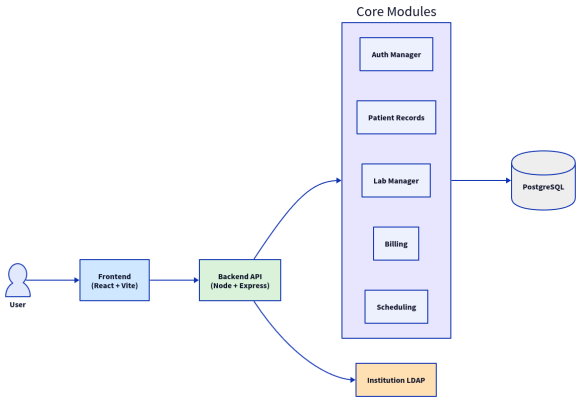

Fig. 1. Overall PHC system architecture retained from the SRS/Assignment 1 design baseline and still aligned with the current backend implementation.

B. Finalized Tech Stack and Architectural Direction

The current technical direction is a layered modular monolith. This was an intentional decision. A microservices architecture would have added deployment and coordination overhead that is difficult to justify for a single-campus PHC.

TABLE I
Finalized Technology Stack

Layer	Technology Choice
Frontend	React + Vite (currently minimal scaffold only)
Backend	Node.js + Express (ESM)
Database	PostgreSQL
ORM	Prisma with generated client
Authentication	JWT sessions; local LDAP-backed bind flow for development, institutional LDAP alignment planned for deployment
Security / Infra	Helmet, CORS controls, role-gated routes, Linux deployment target
Testing	Bruno collection, shell-based smoke suite, CSV result ledger

The architectural pattern followed consistently in the backend is:

$$\text{route} \rightarrow \text{controller} \rightarrow \text{service} \rightarrow \text{Prisma}$$

This discipline is valuable in an academic project because it makes responsibility boundaries explicit. The controllers remain thin, the services hold validation and business rules, and Prisma is the persistence boundary. Such separation reduces the chance that future frontend work or later Sprint 5 integrations will require invasive rewrites.

C. Protocols, Algorithms, and Methodology Finalized

The project does not depend on one mathematically novel algorithm; instead, its value lies in selecting appropriate operational mechanisms and enforcing them consistently.

1) Authentication and Access Protocol: The finalized access model uses JWT for session continuity and role-based authorization at route level. The repository now includes a Dockerized local LDAP server seeded with PHC test users, and authentication is performed through LDAP bind rather than the earlier fallback-only development stub. This is an important implementation correction after Assignment 1 because it moves authentication behavior closer to the intended institutional model. The remaining gap is not local auth correctness, but production-grade alignment with institute infrastructure, deployment hardening, and operational secret handling.

2) Visit-Centric Traceability Model: The central methodology choice is to treat the visit as the anchor of all clinical artifacts. A visit progresses through states such as WAITING, IN_CONSULTATION, COMPLETED, and CANCELLED. Once this state machine exists, prescriptions, lab requests, bills, and vitals no longer float as isolated records.

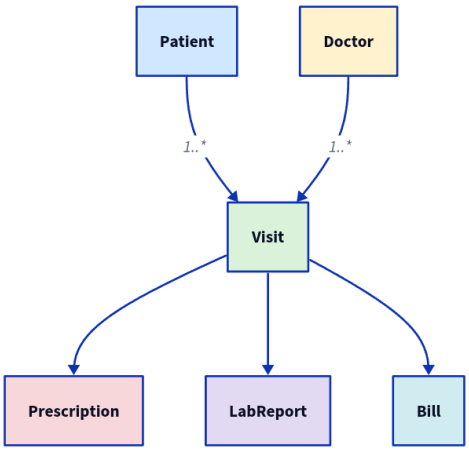

Fig. 2. Visit-centric relational design linking clinical, laboratory, and billing entities.

3) Atomic Operations: Two important implementation choices show methodological maturity beyond the initial report:

- Billing transaction: bill creation and stock deduction are executed atomically using prisma.$transaction([...]).
- QR check-in composition: patient resolution, visit creation, and optional vitals insertion occur as one logical check-in operation.

These are small but significant examples of where the implementation moves from "feature present" to "feature defensible under failure conditions".

4) Agile / Scrum Execution: The SRS defines Agile development with short sprints, Trello-based work management, and Essence Alpha ownership. The repository history and documentation indicate that this process was not treated as a formality; it influenced both the module ordering and the re-scoping of later work.

D. Sprint Plan, Scrum Team, and Alpha Progression

The project is being executed by a three-member Scrum-style team with shared implementation responsibility and Alpha ownership used as an accountability model. From SRS 2.0, the Alpha mapping is as follows.

TABLE II
Essence Alpha Ownership

Alpha	Responsible Member
Stakeholders, Opportunity, Work	Chaitanya Singh
Software System, Team	Yash Bavadiya
Requirements, Way of Working	Nayan Patidar

The original SRS proposed five implementation sprints, but the repository's current planning view extends this into seven execution slices, with frontend and stabilization separated. This is a useful example of theory being corrected by implementation reality.

TABLE III
Observed Sprint Status on April 5, 2026

Sprint	Focus	Status
0	SRS, initial UML, schema framing	Complete
1	Auth, infrastructure, patient profile	Complete
2	Visit lifecycle, doctor attendance and availability	Complete
3	Prescription and laboratory modules	Complete
4	Inventory, billing, appointments, QR check-in, documents	Complete
5	Admin users, events, reports, notifications, LDAP, integration work	Complete
6	React dashboards and UX integration	Not started
7	Hardening, testing, deployment	Not started

Alpha progression is also visible across the project phases:

- Stakeholders and Opportunity: strongest during early PHC workflow study and problem framing.
- Requirements and Software System: dominant during SRS refinement and UML convergence.
- Work and Way of Working: dominant in the current phase, visible through route-by-route delivery, test scripting, and repository conventions.
- Team: increasingly important now that integration, frontend backlog, and documentation synchronization are becoming coordination problems rather than pure design problems.

E. Constraints, Limitations, and Challenges Identified

Several constraints that were abstract in the SRS now appear concretely in the repository:

- Institutional auth remains a deployment concern: local LDAP-backed development is working, but campus-grade deployment integration and operational secrets management still need validation.
- Frontend lag: the backend has become substantial, but the frontend still contains only a minimal React placeholder screen.

- Documentation drift: TESTING.md, smoke_test.sh, and test_cases.csv do not all reflect the same current test count, showing that maintenance of secondary artifacts is now a real project risk.
- Campus-network assumptions: the SRS assumes a stable institutional network and browser/mobile availability, but no offline mode exists.
- Incremental scope pressure: the shift from a five-sprint framing to a seven-sprint README plan reflects the practical need to separate backend completion from frontend delivery and final stabilization.

F. Final UML and Structural Baseline

The final UML baseline used for this report is the same set that emerged from SRS 2.0 and was carried into Assignment 1. This remains a valid choice because the current backend implementation still aligns with those diagrams at a structural level. The actors in the use case model match the implemented role system, the class relationships are visible in the Prisma schema, and the main activity and sequence flows are now reflected by real services and routes.

The component view is especially important. It justified the modular monolith choice and still describes the repository accurately: authentication, records, visits, laboratory, billing, and admin capabilities are separated by module, but deployed as one backend service. That architecture reduced operational overhead while preserving strong internal boundaries.

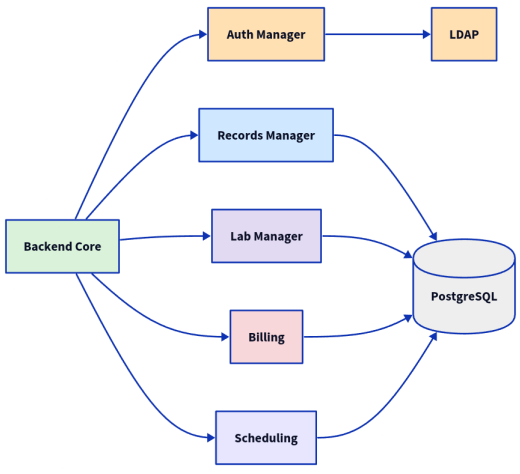

Fig. 3. Component-level decomposition supporting the chosen modular monolith architecture.

III. Results and Observations

A. Implementation Status Achieved

The most important result after Assignment 1 is that the project is no longer only a requirements-and-diagrams exercise. The current repository contains:

- a backend Express application exposing more than fifty documented API routes across authentication, visits, labs, billing, check-in, documents, events, and admin flows,
- a Prisma schema covering identity, visits, prescriptions, lab workflow, pharmacy, billing, attendance, events, and external documents,
- an idempotent seed script for role-specific users,
- a Bruno API collection organized by module,
- an executable smoke test script spanning the implemented backend together with a full end-to-end HTTP flow runner,
- and a minimal frontend scaffold awaiting actual dashboard development.

This asymmetry between backend maturity and frontend immaturity is itself an important result. It shows that the team prioritized the risk-heavy layers first: persistence, business rules, and route security.

B. Implemented Module Coverage

The delivered backend can also be understood as a set of working operational modules rather than only a count of routes. Table IV summarizes the state of the major modules visible in the repository.

TABLE IV
Current Module Coverage

Module	Delivered Capability	Status
Authentication	Login, current-user resolution, JWT guarding, RBAC middleware	Working
Patient records	Self-profile, QR lookup, staff access, visit history	Working
Visit lifecycle	Create, vitals, claim, consultation, complete, cancel, queue	Working
Doctor operations	Availability, attendance, schedule views, doctor listing	Working
Prescription	Create, retrieve, pending queue, dispense	Working
Laboratory	Request tests, pending queue, report upload, patient access	Working
Medicine / billing	Inventory, stock update, bill generation, payment queue	Working
Appointments	Book, list, cancel, emergency override logic	Working
Check-in	QR-based visit creation with optional vitals	Working
Documents	Upload and list external records	Working
Admin / reports / notifications	User management, events, usage and attendance reports, physician-absence notifications	Working
Frontend dashboards	Role dashboards and UX integration	Not started

C. Execution-Time Evidence from Current Artifacts

The testing artifacts contain actual timing evidence rather than only projected estimates. The SRS states

that standard user operations should return within a few seconds under normal load. The measured results satisfy that expectation with substantial margin.

TABLE V
Representative Measured Performance Results

Test ID	Operation	Threshold	Observed
TC-P-001	Login	3000 ms	77 ms
TC-P-002	Patient profile retrieval	3000 ms	233 ms
TC-P-003	Prescription retrieval	3000 ms	1215 ms
TC-P-004	Atomic bill generation	5000 ms	1151 ms
TC-P-005	Atomic QR check-in	3000 ms	796 ms
TC-R-001	Login repeatability (15 runs, avg)	3000 ms	81 ms
TC-R-002	Patient profile repeatability (15 runs, avg)	3000 ms	271 ms
TC-R-003	QR check-in repeatability (15 runs, avg)	3000 ms	889 ms

Across the 48 rows currently recorded in test_cases.csv, the observed overall mean response time is approximately 488.65 ms, with the slowest recorded value at 1291 ms. Even the heavier operations such as billing and prescription retrieval remain well within the SRS's "few seconds" expectation.

D. Positive Results

Several positive findings stand out:

- Traceability is no longer hypothetical. The schema and services now physically enforce visit-linked relationships across the main clinical modules.
- RBAC is operational. Negative test cases show expected denials for unauthorized roles attempting protected actions such as bill generation or pharmacy queue access.
- Atomicity is implemented where it matters. Billing and QR check-in were not left as optimistic multi-step flows.
- Administrative scope expanded successfully. Sprint 5 work includes user management, PHC event publication, reporting, lab-request detail access, patient lab-report retrieval, and in-app doctor-availability notifications.
- Testing maturity improved after Assignment 1. The project now includes a repeatable smoke workflow and explicit timing evidence instead of only projected claims.
- Authentication maturity improved. The project moved from an LDAP design placeholder to a working local LDAP-backed bind flow used by tests and development setup.

E. Negative Results and Corrective Observations

The current state also exposes limits and negative findings that improved the team's understanding of the project:

- Production-grade identity integration is still incomplete. Local LDAP-backed development is working, but real institutional rollout, credential lifecycle, and deployment security still need separate validation.
- Frontend delivery was underestimated. The README still lists Sprint 6 dashboards as not started, and the React application remains a placeholder. This forced a clearer separation between backend completion and user-interface completion.
- Project artifacts need synchronization discipline. The test guide mentions 53 cases, the smoke suite enumerates 56 identifiers, and the CSV ledger records 48 measured rows. This does not invalidate the backend, but it demonstrates a real documentation-maintenance problem.
- Operational quality now depends on integration, not just coding. The remaining blockers are less about writing another route and more about closing gaps among LDAP, staging migration, end-to-end validation, and frontend consumption.

F. Artifact Consistency as a Project Result

An unexpected but important result is that the project now contains enough engineering surface area for artifact consistency itself to matter. The available documents and scripts do not all describe the same testing state.

TABLE VI
Observed Documentation and Testing Drift

Artifact	Observed Count / Claim	Interpretation
TESTING.md	states 53 automated cases	partially updated guide
test_cases.csv	contains 48 measured rows	recorded evidence subset
smoke_test.sh	enumerates 56 test IDs	latest executable scope

This observation is valuable because it reveals a practical engineering challenge that was not obvious at the start of the project: once the system has enough working parts, keeping the surrounding evidence synchronized becomes part of the deliverable.

G. Requirement-to-Implementation Traceability

The project now has enough implemented surface area for a compact traceability view to be meaningful. Table VII summarizes how a few high-value requirements now map to concrete delivered behavior.

TABLE VII
Compact Traceability Matrix

REQ	Delivered Behavior	Implementation Anchor	Sprint
REQ-6	LDAP-oriented authentication design	Auth service / config	1
REQ-15	QR-based patient identification	Patient QR lookup, check-in route	4
REQ-22	Claimed consultation ownership	Visit claim service	2
REQ-28	Visit-linked traceability	Prisma visit relations	2
REQ-35/38	Doctor attendance and checkout hours	Doctor attendance module	2
REQ-44/46	Prescription creation and dispense flow	Prescription routes and services	3
REQ-54/55	Lab request and report upload	Lab routes and services	3
REQ-64/66	Billing plus inventory update	Billing transaction, medicine stock	4
REQ-49/52	Admin user and reporting capability	Admin routes and report service	5

H. Module-Level Timing Interpretation

The CSV evidence also makes it possible to compare average response behavior by module. This is useful because it distinguishes lightweight identity and profile operations from the heavier composite workflows that traverse more relations and business rules.

TABLE VIII
Selected Module-Wise Average Response Times from test_cases.csv

Module	Average	Interpretation
Auth	171.0 ms	lightweight identity operations
Patient	263.2 ms	profile retrieval and updates remain fast
Visit	310.4 ms	queue and lifecycle actions are efficient
Billing	662.29 ms	heavier due to transaction and stock logic
Checkin	680.75 ms	patient resolution plus visit creation work
Prescription	748.57 ms	relation-heavy clinical retrieval and creation
Lab	740.0 ms	upload and request operations are moderately heavy
Appointment	586.75 ms	validation-heavy but still sub-second

I. Theory Versus Implementation Corrections

One of the strongest outcomes of the project is that several theoretical assumptions from the earlier stages were refined after implementation:

1) Sprint Structure: The SRS proposed five implementation sprints. The current README expands the plan into seven slices, separating frontend and stabilization into dedicated later phases. This correction improved realism. It acknowledges that "system integration" in a healthcare workflow is not equivalent to "system finished".

2) Performance Expectations: The SRS described performance qualitatively. The actual backend now shows sub-1.3-second behavior across recorded measurements, allowing the team to replace vague assumptions with concrete evidence. In particular, the QR check-in flow, which was expected to materially reduce reception effort, is measured at 796 ms in one NFR run and 889 ms average across repeatability trials.

3) Data-Model Confidence: The earlier report argued that a relational model was preferable for PHC traceability. The implemented Prisma schema strengthens that conclusion. Prescriptions, lab reports, bills, vitals, doctor attendance, and documents now coexist in a model where referential integrity is inspectable and testable rather than only architecturally claimed.

J. Operational Factors Identified Late

Some debilitating factors became clear only after implementation work started:

- Development conveniences can still hide deployment risk. Even though local LDAP-backed development is now working, institute-grade rollout, credential lifecycle handling, and deployment security still require separate validation.
- Testing evidence itself becomes an artifact to maintain. Once the project gained automated checks, consistency between scripts, CSV logs, and narrative docs became necessary.
- User-interface absence limits stakeholder validation. Even with a working backend, a PHC stakeholder will still perceive the product as incomplete until role dashboards exist.

K. Novelty and Comparative Advantage

The project should not overclaim against the global state of the art; no formal benchmark against commercial hospital platforms has been conducted. However, relative to generic appointment tools and heavyweight hospital systems, the design does exhibit practical novelty in its target context:

- Institution-specific identity integration: the design is anchored around IIT Jodhpur credentials rather than creating a separate credential universe.
- Unified outpatient traceability: check-in, consultation, prescription, lab, billing, and documents are handled in one visit-centric model.
- Lightweight deployability: a modular monolith is better matched to a campus PHC than an enterprise hospital stack or a fragmented set of standalone tools.
- Administrative observability: attendance, reports, and event management are treated as first-class system concerns rather than afterthoughts.

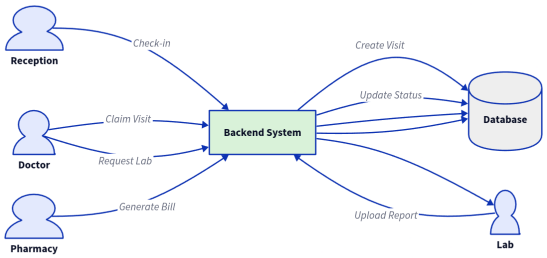

Fig. 4. End-to-end patient-flow model retained as a valid summary of the working backend's intended operational path.

L. Current Project Position

As of April 5, 2026, the clearest defensible conclusion is that the PHC Integrated Digital System has crossed the line from conceptual architecture to a functionally rich backend platform. Sprint 5 backend integration is effectively closed, and the remaining risks are now concentrated in frontend completion, integration hardening, documentation consistency, and institutional deployment details rather than in basic system design.

IV. Discussion

A. Security and Privacy Implications

Healthcare systems must be evaluated not only by functionality but by whether access boundaries are enforceable. The current backend already demonstrates several positive security-oriented properties: authenticated access for non-public routes, role-gated authorization, centralized error handling, response normalization, and request-surface protection through middleware such as helmet.

At the same time, the repository clarifies what is still pending before a production interpretation would be justified. Stronger auditability, HTTPS deployment enforcement, institutional identity rollout, and operational secrets discipline belong to the remaining hardening work. This distinction is important because the project already reflects a secure design direction even though the deployment-grade security story is incomplete.

B. Why the Chosen Design Still Holds

The implemented repository strengthens the original argument for a modular monolith. A three-member team was able to deliver a broad backend, preserve module boundaries, and maintain testability without the operational complexity of distributed services. The design therefore appears well matched to an institute PHC where maintainability and low deployment friction matter more than premature service decomposition.

C. What Assignment 2 Changed in Team Understanding

The most important change since Assignment 1 is epistemic: the project is no longer judged only through diagrams and projected benefits. It now produces empirical feedback. Route behavior, timing measurements, negative-case failures, unfinished integration points, and documentation drift all provide concrete evidence that shapes the next iteration. That shift from modeled confidence to observed confidence is one of the strongest educational outcomes of the project.

V. Conclusion

The Assignment 2 state of the PHC Integrated Digital System shows meaningful convergence between requirements, architecture, and implementation. The team has preserved the strongest ideas from Assignment 1 and SRS 2.0, especially role-based access, visit-linked traceability, QR-assisted reception flow, and modular separation of concerns, while also correcting earlier assumptions through concrete implementation work.

The results are mixed in the right way. The backend demonstrates substantial functional progress, working LDAP-backed development authentication, and encouraging performance behavior, but the project also reveals absent production-grade frontend dashboards and the maintenance burden of keeping testing and documentation artifacts synchronized. These are not signs of conceptual failure; they are signs that the project has moved into the integration-heavy phase where disciplined engineering matters more than broad ideation.

For the PHC context at IIT Jodhpur, the project is already strong as a backend-centered digital foundation. The next decisive milestone is to close the loop among frontend usability, institutional authentication, end-to-end validation, and deployment readiness.

VI. AI Usage Declaration

AI tools were used only for writing assistance, restructuring, and language refinement of this report. The technical basis of the report was derived from the repository contents, SRS 2.0 / 2.1 / 2.2, Assignment 1 artifacts, UML diagrams, and current implementation files. Architectural decisions, sprint structure, requirements framing, and implementation work remain the team's own project contributions.

References

[1] Y. Bavadiya, C. Singh, and N. Patidar, "Software Requirements Specification for PHC Integrated Digital System, Versions 2.0, 2.1, and 2.2," IIT Jodhpur, Feb.–Apr. 2026.
[2] Y. Bavadiya, C. Singh, and N. Patidar, "Design and Implementation of the PHC Integrated Digital System: A Modular Healthcare Architecture for IIT Jodhpur," Assignment 1 submission, IIT Jodhpur, Feb. 2026.
[3] "PHC Integrated Digital System README," repository artifact, Mar. 2026.
[4] "PHC System – Testing Guide," repository artifact, Mar. 2026.
[5] "PHC Integrated Digital System automated smoke test suite," backend/smoke_test.sh, repository artifact, Mar. 2026.

[6] "PHC Integrated Digital System test case ledger," test_cases.csv, repository artifact, Mar. 2026.
[7] IEEE Computer Society, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.
[8] K. Schwaber and J. Sutherland, "The Scrum Guide," Scrum.org, Nov. 2020. [Online]. Available: https://scrumguides.org/
[9] PostgreSQL Global Development Group, "PostgreSQL Documentation." [Online]. Available: https://www.postgresql.org/docs/
[10] Prisma Data, Inc., "Prisma ORM Documentation." [Online]. Available: https://www.prisma.io/docs/orm
[11] OpenJS Foundation, "Express – Node.js web application framework." [Online]. Available: https://expressjs.com/
[12] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015.
[13] OWASP Foundation, "OWASP Application Security Verification Standard 4.0.3," 2021. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

Appendix A
Final UML Diagrams

The appendix reproduces the UML set carried forward from the SRS/Assignment 1 design baseline. These diagrams remain the best compact visual summary of the intended end-to-end system behavior and structural decomposition. If the course staff expects newer diagram revisions, insert them here in place of the inherited versions.

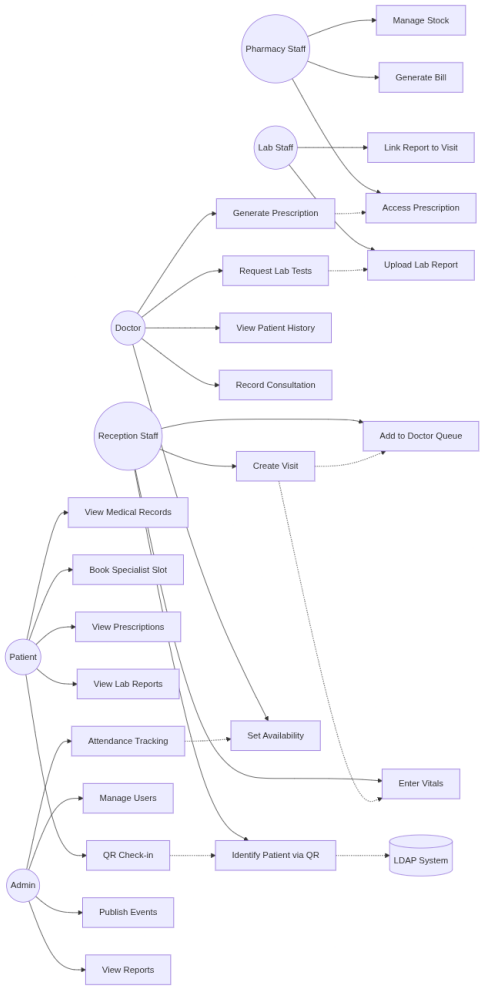

Fig. 5. Use case diagram.

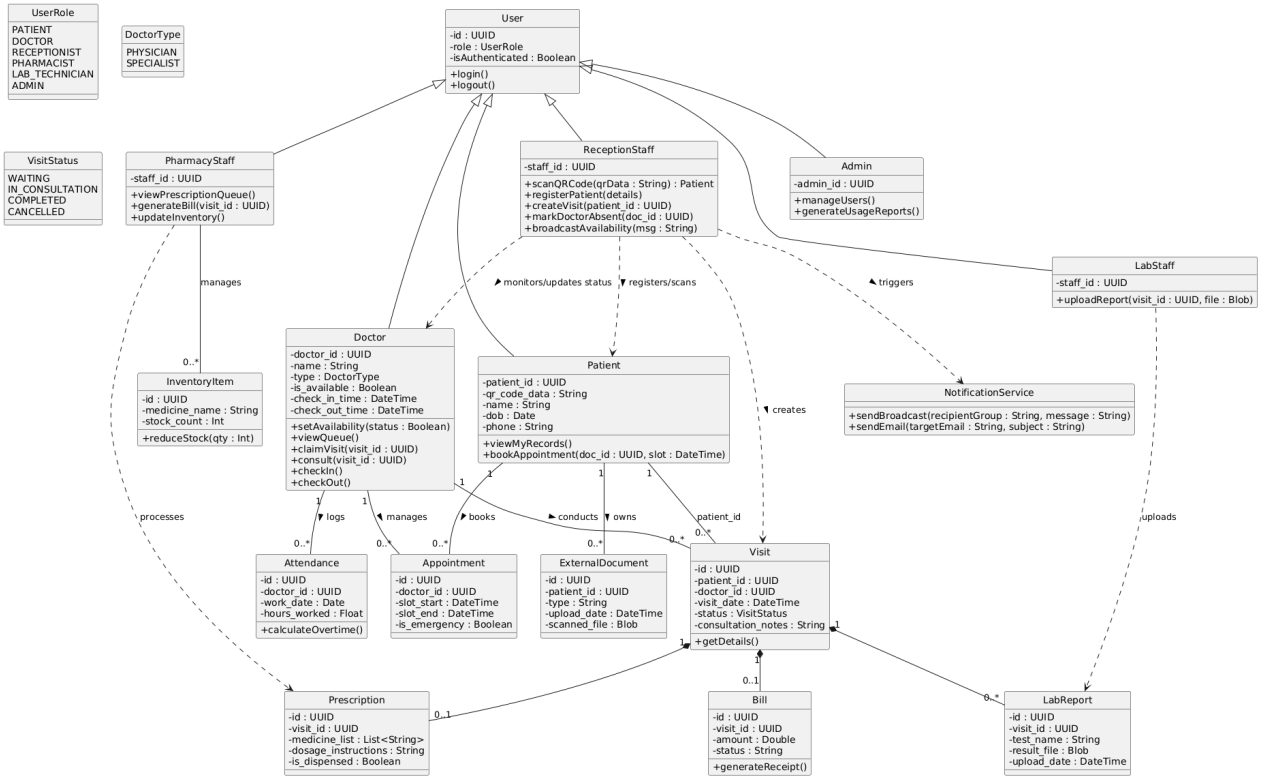

Fig. 6. Class diagram.

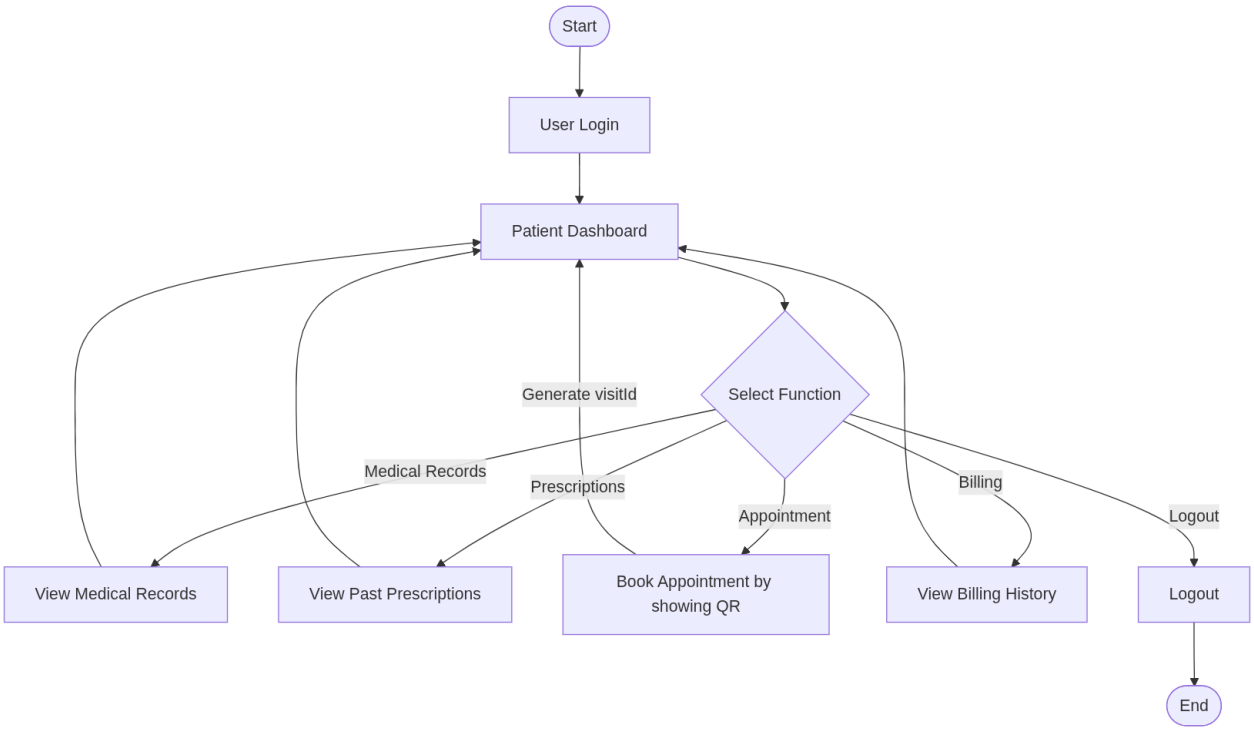

Fig. 7. Patient workflow activity diagram.

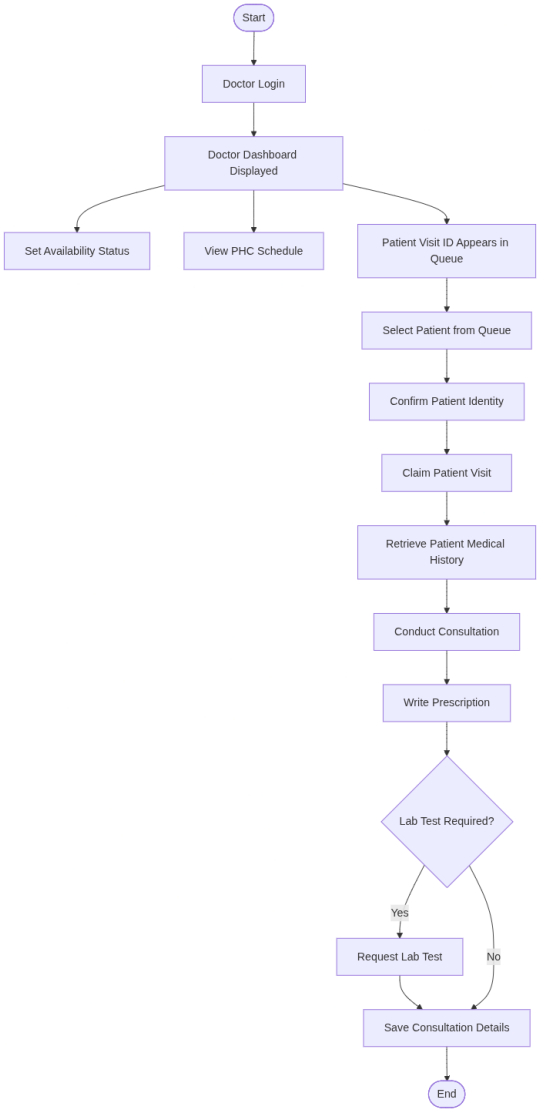

Fig. 8. Doctor workflow activity diagram.

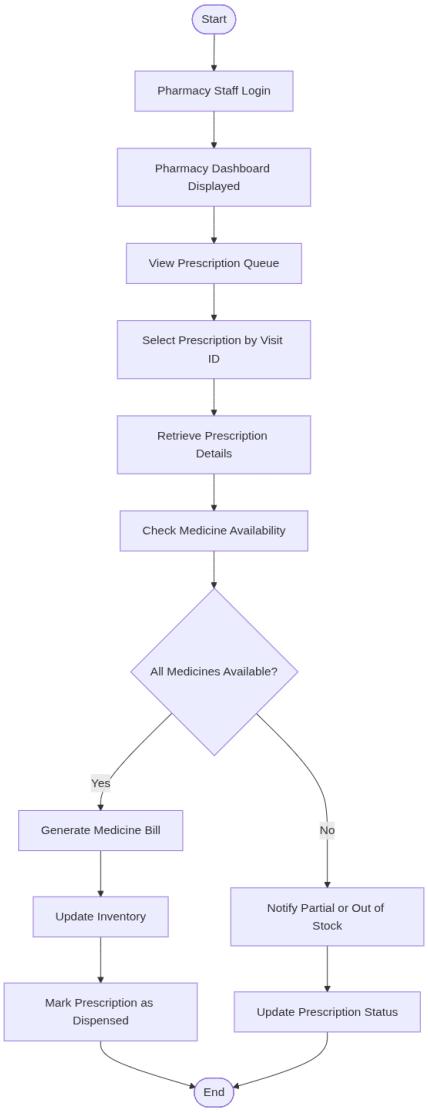

Fig. 9. Pharmacy workflow activity diagram.

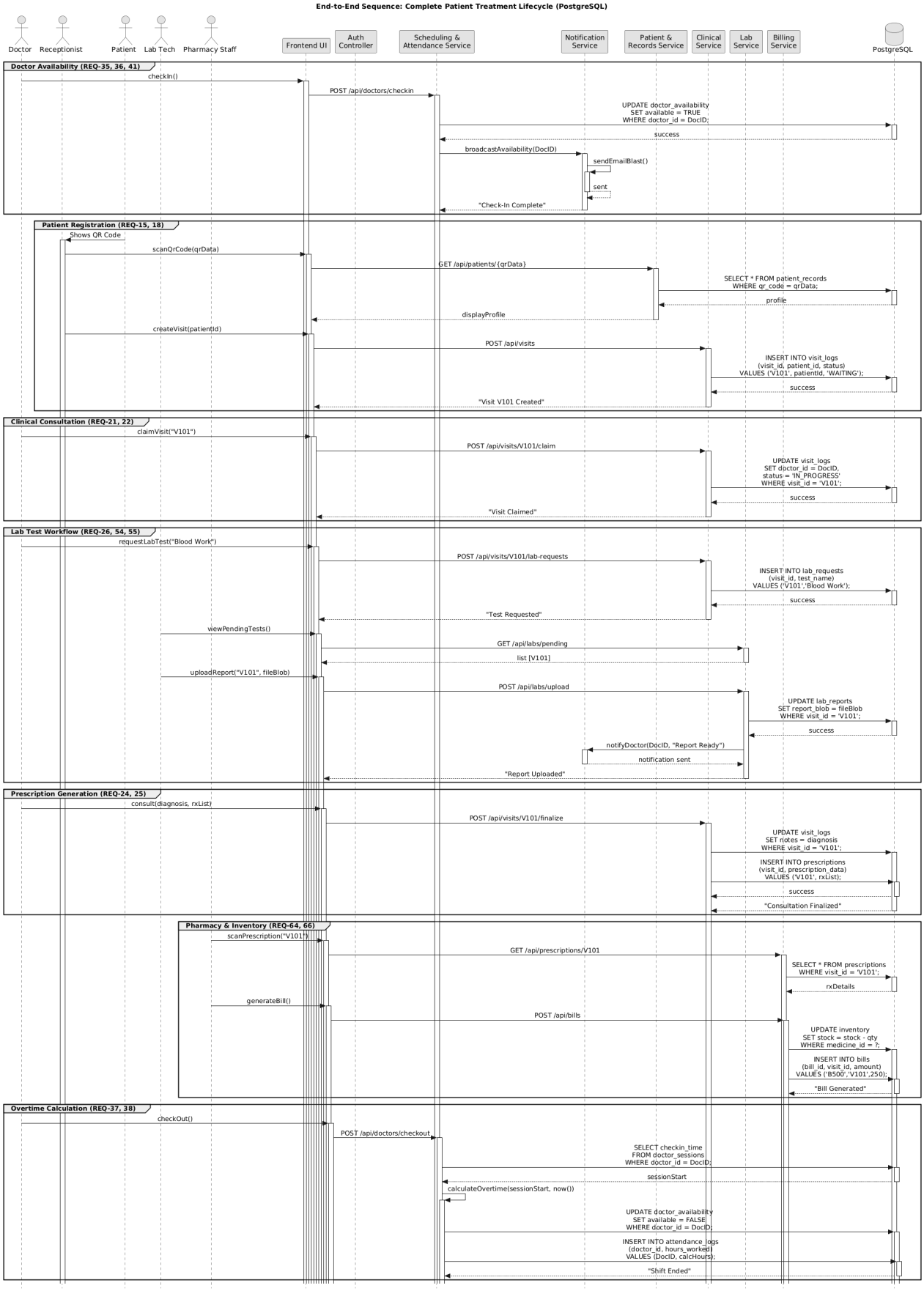

Fig. 10. Sequence diagram.